

Report

Part 1

We were asked to develop 4 java classes which are `Model`, `Algorithm`, `SupervisedLearner` and `TestLearner` (which contains the main method to see if the implemented code works as intended). Also, for this Lab we had to add a new method `augment` in the `Vector` class to include the bias when computing gradient in the `Algorithm` class. The class `Model` contains 1 private attribute: `params`, which is an instance of class `Vector` that represents the parameter vector which contains coefficients of that model, and it gets updated after each analysed record from the dataset. Apart from the constructor method, there are 3 methods: `getParams`, `predict` (which uses the param vector to compute the dot product), `update` (updates the parameter vector according to the learning rate and the gradient descent given as arguments). `Algorithm` contains 2 private attributes: `learning_rate` (constant learning rate which defines how far/fast the gradient descent moves) and `stopping_criterion` (constant which defines the minimum difference between the correct linear regression and model's prediction of parameter vector). Also, apart from the constructor method, there are 2 methods: `gradient` (calculates how far our parameter vector is from the solution) and `solve` (given a dataset uses gradient descent to return a model passed through supervised learning and stabilized basing on stopping criterion). Furthermore, the class `SupervisedLearner` embraces the classes described above (and `Dataset`) to perform the full process of linear regression using supervised learning. It has 3 attributes: `algorithm` (determines what algorithm will be used to teach the model), `dataset` (on which the model is trained) and `model` (already trained model ready to be used for new input vectors, once `solve` method has been executed). Apart from the constructor method, there are 3 methods: `solve` (creates a model which was trained on given dataset with given algorithm), `predict` (use trained model to predict new input vectors) and `toString` (same as in `Vector` class). Finally, `TestLearner`, is the main function which uses the methods of the other classes to check their functionality.

Part 2

In this Lab, no alternative solutions were discussed. And that is because it was mostly based on implementing the given formulas. When it comes to related theoretical concepts, it can be remarked the composition relation from `Model` to `SupervisedLearner`, since a `Model` cannot exist by itself, but an instance of it is output in the `solve` method of `Algorithm` and then attributed in the `solve` method of `SupervisedLearner`.

Part 3

When it came to testing the correct implementation of the classes, most of the methods could be tested by checking that the prediction of instance `SupervisedLearner` trained on specific dataset is close enough to the expected value. That is because the use of the `solve()` method implies the use of all the methods and constructors in the other classes. Therefore, if the test proved the correct implementation of `sl.solve()`, which indeed proved it, we were done.

When implementing the `TestLearner`, we found out that the dataset should be bigger than 2 records and it has to have a variety of input combinations, therefore, we had to extend the dataset on which `SupervisedLearner` was trained.